Software Requirements Specification

SmartSchool — Student Records, Fee Invoicing & Staff Payroll Management System

Prepared for: **ICT University Yaóundé**

Course: **SEN4121 – Large System Environment** (Practical Examination, Summer 2026)

Examiner: Engr. Tanwi Nkiamboh

Document version:	1.0
Group number:	Group 2
Date:	August 17, 2026
Status:	Living document – updated through Week 4

Group Member	Matricule
Kwete Rayan	ICTU20241377
Kamdeu Yamdjeuson Neil Marshall	ICTU20241386
Younga Tchappi Dylane	ICTU2241836
Fru Chi Ehud Neba	ICTU20241812

This document specifies the functional and non-functional requirements for SmartSchool, a microservice-based ERP for a school, covering student academic records, fee invoicing, and staff payroll, built as part of the SEN4121 6-week practical examination series.

Contents

Chapter 1
Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for SmartSchool, a school Enterprise Resource Planning (ERP) system built as a set of cooperating microservices behind a single API Gateway. It restates the system's required behaviour as verifiable requirements, independent of implementation detail, and is intended as an agreed basis between the development group and the course examiner for what the system must do across the six weekly practical sessions of SEN4121.

Unlike a purely aspirational specification, every requirement below is tagged with a **status**: `Implemented` (already built and demonstrable) or `Planned` (required by a later week of the exam series but not yet built). This keeps the document honest about the current state of the system rather than describing a system that does not yet exist.

1.2 Scope

SmartSchool covers the three functional areas required by the exam brief:

- **Academic** – student records, courses, enrollments, and grades.
- **Finance** – fee invoicing, automatically triggered by enrollment.
- **HR / Payroll** – staff records and salary payslips.

The system serves three internal actors – **Admin**, **Teacher**, and **Student** – through a React single-page application, all traffic passing through a single API Gateway. There is currently no external payment-gateway integration: invoices carry a mobile-money reference field for future reconciliation, but automated payment collection is out of scope for the current iteration (see Section 2.5).

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
Microservice	A small, independently deployable service responsible for one job (e.g. `auth-service` only handles identity).
API Gateway	The single entry point (`gateway`) that receives every external request and forwards it to the correct backend service.
RBAC	Role-Based Access Control – what a user may do depends on their role (`admin`, `teacher`, `student`).
JWT	JSON Web Token – a signed token issued at login that proves identity on later requests without a database lookup.
ERD	Entity-Relationship Diagram – shows database tables, columns, and how they connect.

Term	Definition
3NF	Third Normal Form – a database design rule: no needless duplication, every column depends only on its table's primary key.
Message Broker	Middleman software (RabbitMQ) that lets one service publish an event another service consumes later, without a direct call.
Asynchronous Workflow	A process where the publishing service does not wait for the consuming service to finish.
OWASP Top 10	The ten most common web application security risks.
Load Testing	Sending many concurrent requests to observe system behaviour under pressure.
CI/CD	Continuous Integration / Continuous Deployment – automated build and test on every push.
SRS	Software Requirements Specification (this document).
SDD	Software Design Document – the companion document describing architecture and diagrams.
FR / NFR	Functional Requirement / Non-Functional Requirement.

1.4 References

- SmartSchool Software Design Document, Version 1.0, Group 2, SEN4121, ICT University Yaóundé.
- SEN4121 Large System Environment – Practical Examination brief, Summer 2026, Engr. Tanwi Nkiamboh.
- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications (structural reference for this document).
- Project source repositories: `MarketFlow` (backend services, database, documentation) and `marketflow_frontend` (React SPA).

1.5 Document Overview

Chapter 2 gives an overall description of the product, its actors, and constraints. Chapter 3 presents specific requirements: external interfaces and functional requirements grouped by module, each tagged with implementation status. Chapter 4 presents non-functional requirements. Chapter 5 lists data requirements derived from the relational schema. Chapter 6 traces requirements to the SDD and to the exam's weekly mark scheme.

Chapter 2
Overall Description

2.1 Product Perspective

SmartSchool is a new, self-contained school-management platform composed of cooperating services, all reached through one API Gateway:

Component	Responsibility
Gateway	Single public entry point; JWT verification, routing by URL prefix, rate limiting, combined Swagger UI.
Auth Service	Registration, login, JWT issuance, and the shared staff/user identity store.
Academic Service	Courses, enrollments, and grades; publishes an event when a student enrolls.
Finance Service	Invoices; consumes the enrollment event and creates an invoice asynchronously.
MySQL Database	Shared relational store for all modules (`smartschool` schema).
RabbitMQ	Message broker carrying the `enrollment.created` event between Academic and Finance.
React Frontend	Single-page application for registration, login, and role-scoped dashboards.

Only the Gateway is reachable from outside the Docker network; every backend service is internal-only, reflecting the "API Gateway as the only front door" principle required by the exam brief.

2.2 Product Functions

At a high level, SmartSchool shall allow the system to:

1. Register and authenticate Admins, Teachers, and Students, issuing a role-carrying JWT on login.
2. Let an Admin manage courses and view system-wide reports.
3. Let a Student browse courses, enroll, view their own enrollments, grades, and invoices.
4. Let a Teacher record grades for the courses they teach and view those grades.
5. Automatically generate a fee invoice whenever a student enrolls in a course, without the enrollment request waiting on it.
6. Enforce role-based access at both the Gateway ("is this a valid, logged-in user?") and each service ("is this specific action allowed for this role/owner?").
7. Protect the API from abuse via per-IP rate limiting.
8. Compute and record staff payroll (*planned, see Section 2.5*).

2.3 User Classes and Characteristics

User Class	Characteristics
Admin	Highest-privilege internal user. Creates courses, enrolls students on their behalf, views reports, and (planned) runs payroll. Assumed to have elevated authorization and periodic use.
Teacher	Records grades only for enrollments in courses they teach; can view system reports alongside Admin. Assumed to be a member of staff with routine, course-scoped use.
Student	Registers, logs in, browses/enrolls in courses, and views their own enrollments, grades, and invoices. Assumed to have basic digital literacy.

2.4 Operating Environment

SmartSchool is a distributed, container-based system: every backend service, the MySQL database, and RabbitMQ run as Docker containers orchestrated by a single `docker-compose.yml`, brought up with one command (`docker compose up --build`). The React frontend is a separate containerized application that talks to the Gateway over HTTP from the browser. No specific host operating system is required beyond a working Docker Engine.

2.5 Design and Implementation Constraints

- All inter-service authentication relies on a single shared JWT signing secret (`JWT_SECRET`), distributed to every service via environment variable – acceptable for a course project, not for a production multi-team deployment.
- The Gateway performs authentication ("is this a valid token?") only; authorization (role/ownership checks) is deliberately left to whichever service owns the resource.
- Stock/academic domain rule: a course fee is captured on the `courses` table and *snapshotted* onto the generated invoice at enrollment time, so a later fee change does not retroactively alter an already-issued invoice.
- **Payment collection is out of scope for the current iteration.** An invoice's `momo_reference` and `method` columns exist in the schema for a future mobile-money reconciliation flow, but no external payment gateway is called by the system today; invoices are created with status `pending` and there is currently no endpoint to mark one `paid`.
- **Payroll is schema-only.** The `payroll_runs` and `payslips` tables exist in the database design (Week 2) to satisfy the HR module's data model, but no service currently exposes payroll endpoints; this is planned as a Week 5 module feature (see Chapter 3, HR requirements).

2.6 Assumptions and Dependencies

- Docker Engine and Docker Compose are available on the host running the system.
- RabbitMQ and MySQL containers reach a healthy state before dependent services start (enforced via Compose health checks).
- The frontend and Gateway are reachable from the same browser origin context (or CORS is permissive, see NFR-SEC-3).

- Course fee amounts and role assignments are configured by an Admin before students enroll.

Chapter 3
Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- A registration/login screen for all three roles.
- A role-aware dashboard: every authenticated user sees the same shell, but the actions available (e.g. creating a course, recording a grade) are gated by role both in the UI and, authoritatively, by the backend.
- Live demo panels on the dashboard that call `/reports`, `/courses`, `/enrollments`, and `/finance/invoices` through the Gateway and render the raw JSON response, so the request/response cycle is directly observable during the oral evaluation.

3.1.2 Hardware Interfaces

None. SmartSchool is a purely software, browser/server-based system; no specialized hardware (POS terminals, barcode scanners, biometric devices) is required.

3.1.3 Software Interfaces

Interface	Description
MySQL 8	Relational store for all modules; accessed by each service via a pooled `mysql2` connection using parameterized queries.
RabbitMQ 3.13 (AMQP 0-9-1)	Direct exchange `school_events`; carries the `enrollment.created` event from Academic Service to Finance Service.
Docker / Docker Compose	Packaging and orchestration for every backend service plus the database and broker.
OpenAPI 3 / Swagger UI	Combined API documentation for every Gateway-routed endpoint, served at `GET /docs`.

3.1.4 Communications Interfaces

All external communication (frontend, curl, Postman) to the system happens over HTTP/REST with JSON bodies, through the Gateway, authenticated with a `Bearer` JWT in the `Authorization` header where required. Internal communication between Academic Service and Finance Service for the enrollment/invoice workflow happens asynchronously over AMQP via RabbitMQ, not direct HTTP.

3.2 Functional Requirements

Each requirement below is tagged **[Implemented]** (built and demonstrable today) or **[Planned]** (required by a later exam week, not yet built).

3.2.1 Account, Authentication and Access Control

ID	Requirement	Status
FR-AUTH-1	The system shall allow a new user to register with a name, email, and password, and an optional role (`admin`, `teacher`, or `student`, defaulting to `student`).	Implemented
FR-AUTH-2	The system shall store the password only as a bcrypt hash, never in plain text.	Implemented
FR-AUTH-3	The system shall allow a registered user to log in with email and password and, on success, issue a JWT carrying the user's id, email, and role, expiring after 1 hour.	Implemented
FR-AUTH-4	The system shall allow a client to retrieve the identity encoded in its own token via `GET /api/v1/auth/me`.	Implemented
FR-AUTH-5	The system shall never include the password hash in any API response.	Implemented
FR-AUTH-6	The Gateway shall verify the signature and expiry of every protected request's JWT before forwarding it to a backend service; an invalid or missing token shall be rejected with `401` at the Gateway, without reaching a backend service.	Implemented
FR-AUTH-7	Each backend service shall independently re-verify the JWT and enforce role/ownership rules specific to the resource it owns (defense in depth), rather than trusting the Gateway's check alone.	Implemented

3.2.2 Academic Module – Courses

ID	Requirement	Status
FR-ACAD-1	The system shall allow any authenticated user to list all courses.	Implemented
FR-ACAD-2	The system shall allow any authenticated user to retrieve a single course by id.	Implemented
FR-ACAD-3	The system shall allow only an Admin to create a new course, including its code, name, assigned teacher, credits, fee amount, and term.	Implemented

3.2.3 Academic Module – Enrollments

ID	Requirement	Status
FR-ACAD-4	The system shall allow a Student to enroll themselves in a course.	Implemented
FR-ACAD-5	The system shall allow an Admin to enroll a specified student in a course on their behalf.	Implemented
FR-ACAD-6	The system shall reject a duplicate enrollment (the same student in the same course) with a clear conflict response, rather than creating a second row.	Implemented

ID	Requirement	Status
FR-ACAD-7	The system shall restrict a Student's view of enrollments to their own records only; Admin and Teacher may view or filter by student or course.	Implemented
FR-ACAD-8	On a successful enrollment, the system shall publish an `enrollment.created` event to the message broker *after* the enrollment is durably committed to the database, without the enrollment response waiting on the event being consumed.	Implemented

3.2.4 Academic Module – Grades

ID	Requirement	Status
FR-ACAD-9	The system shall allow an Admin or Teacher to record a grade (quiz, assignment, midterm, or final) for a given enrollment, with a score between 0 and 100.	Implemented
FR-ACAD-10	The system shall restrict a Teacher to recording grades only for enrollments in courses they themselves teach, rejecting other attempts with `403`.	Implemented
FR-ACAD-11	The system shall restrict a Student's view of grades to their own; a Teacher shall see only grades for courses they teach; an Admin shall see all grades.	Implemented

3.2.5 Finance Module – Invoicing

ID	Requirement	Status
FR-FIN-1	Upon consuming an `enrollment.created` event, the system shall create exactly one invoice for that enrollment, using the course's fee amount at the time of enrollment as a price snapshot.	Implemented
FR-FIN-2	If the same enrollment event is delivered more than once (at-least-once broker delivery), the system shall not create a duplicate invoice; the duplicate is acknowledged as a successful no-op.	Implemented
FR-FIN-3	The system shall allow a Student to list only their own invoices, and an Admin/Teacher to list all invoices or filter by student.	Implemented
FR-FIN-4	The system shall record a mobile-money/transaction reference field on each invoice for future reconciliation.	Implemented (field only)
FR-FIN-5	The system shall provide a way to mark an invoice as paid and reconcile it against a mobile-money or online payment gateway.	Planned (Wk 5/6)

3.2.6 HR Module – Staff and Payroll

ID	Requirement	Status
FR-HR-1	The database shall maintain a payroll run record (period start/end, status) and one payslip per staff member per run (base salary, deductions, net pay).	Implemented (schema only)

ID	Requirement	Status
FR-HR-2	The system shall allow an Admin to open a payroll run for a given period and compute each staff member's net pay as gross pay minus deductions.	Planned (Wk 5)
FR-HR-3	The system shall send computed payouts to staff via a payment channel and record the resulting reference on each payslip.	Planned (Wk 5/6)
FR-HR-4	The system shall confirm payroll completion to the Admin with a summary once processing finishes.	Planned (Wk 5)

3.2.7 Gateway and Cross-Cutting Requirements

ID	Requirement	Status
FR-GW-1	The system shall expose exactly one publicly reachable entry point (the Gateway); no backend service shall be directly reachable from outside the container network.	Implemented
FR-GW-2	The Gateway shall route each external request to the correct backend service based on its URL prefix (e.g. `/api/v1/courses` → Academic Service).	Implemented
FR-GW-3	The Gateway shall limit each client IP to a configurable maximum number of requests per time window (default: 10 requests / 60 seconds) across all routes except `/health`, responding `429` once exceeded.	Implemented
FR-GW-4	The system shall publish machine-readable API documentation (OpenAPI/Swagger) for every Gateway-routed endpoint, viewable at `/docs`.	Implemented

3.2.8 Reporting

ID	Requirement	Status
FR-RPT-1	The system shall allow an Admin or Teacher to view an academic performance report; a Student shall be denied access to this report with `403`.	Implemented
FR-RPT-2	The role check on the reporting endpoint shall be changeable (e.g. Admin-only) without changing the authentication mechanism, and shall take effect on service restart.	Implemented

3.3 Use-Case-Driven Requirement: Enroll in a Course

This requirement is elaborated separately because it drives the system's central asynchronous, event-based workflow and is the most architecturally significant use case in SmartSchool.

Field	Description
Primary actor	Student (or Admin, enrolling on a student's behalf)
Preconditions	Actor is authenticated with a valid JWT; the target course exists.

Field	Description
Main flow	(1) Actor submits `POST /api/v1/enrollments` with a course id through the Gateway; (2) the Gateway verifies the JWT and forwards the request to Academic Service; (3) Academic Service creates the enrollment row and commits it to MySQL; (4) Academic Service publishes an `enrollment.created` event to RabbitMQ and immediately returns `201` to the actor, without waiting on step 5; (5) Finance Service, independently and at its own pace, consumes the event and creates a pending invoice snapshotting the course's current fee.
Alternate flow	If the student is already enrolled in the course, the system returns `409 Conflict` and no event is published. If RabbitMQ or Finance Service is temporarily unavailable, the enrollment still succeeds; the event remains durably queued and the invoice is created once the consumer reconnects.
Postconditions	The enrollment exists and is visible to the student; an invoice for that enrollment eventually appears in the student's "My invoices" view, with no further action required from the actor.

Chapter 4
Non-Functional Requirements

4.1 Security

NFR-SEC-1 Every database query, in every service, shall use parameterized statements; no request-supplied value shall ever be concatenated directly into a SQL string (mitigates OWASP A03: Injection).

NFR-SEC-2 Passwords shall be hashed with bcrypt (minimum 10 salt rounds) before storage; JWTs shall carry a signed expiry of no more than 1 hour (mitigates OWASP A07: Identification and Authentication Failures).

NFR-SEC-3 Password hashes shall never appear in any API response (mitigates OWASP A02/A04: Cryptographic Failures / Insecure Design around sensitive data exposure). CORS is currently permissive across services; this is an accepted, documented gap because the system is a Bearer-token API with no ambient cookie credential to abuse.

NFR-SEC-4 The Gateway's rate limiter shall apply to authentication routes as well as data routes, so it doubles as brute-force login throttling without dedicated code.

NFR-SEC-5 Role checks shall be enforced at the service that owns the resource, not only at the Gateway, so that a compromised or bypassed Gateway check cannot alone grant unauthorized access.

4.2 Performance and Scalability

NFR-PERF-1 Under a load test of at least 20 concurrent virtual users against the Gateway, the 95th-percentile response time for successful (non-rate-limited) requests shall remain under 500 ms.

NFR-PERF-2 The rate limit window and maximum shall be externally configurable (`RATE_LIMIT_WINDOW_MS`, `RATE_LIMIT_MAX`) without a code change, to support both a "safety" profile (default limits, demonstrates throttling) and a "speed" profile (raised limits, measures genuine backend latency) during load testing.

NFR-PERF-3 Academic Service and Finance Service shall be independently deployable and scalable processes, so that load on one module does not require scaling the other.

4.3 Reliability and Consistency

NFR-REL-1 The enrollment-to-invoice workflow shall tolerate a temporarily unavailable message broker or Finance Service without failing or blocking the enrollment request (see FR-ACAD-8).

NFR-REL-2 Event messages shall be published and consumed as durable/persistent so that a broker restart does not silently lose an in-flight enrollment event.

NFR-REL-3 The event consumer shall be idempotent with respect to redelivery: processing the same event twice shall not create a duplicate invoice (see FR-FIN-2).

NFR-REL-4 Each service shall retry its connection to MySQL/RabbitMQ with backoff on startup, so that container start-order races do not cause a permanent failure.

4.4 Maintainability and Portability

NFR-MAINT-1 The entire backend (database, broker, and all services) shall start from a single command (`docker compose up --build`) against a clean environment.

NFR-MAINT-2 Configuration (database credentials, JWT secret, service URLs, rate-limit parameters) shall be supplied via environment variables, never hard-coded.

NFR-MAINT-3 Each service shall own its own Dockerfile and be independently buildable and runnable in isolation for local development.

4.5 Usability

NFR-USE-1 Every Gateway-routed endpoint shall be documented in a single, interactive Swagger UI reachable at one URL.

NFR-USE-2 The frontend shall clearly display the signed-in user's role and shall reflect role-based restrictions in what actions are offered, in addition to the backend's authoritative enforcement.

4.6 Auditability

NFR-AUD-1 Every published domain event shall carry a unique event id, an event type, and an ISO-8601 occurrence timestamp, to support later tracing of what happened and when.

NFR-AUD-2 Every invoice shall be traceable to exactly one enrollment via a unique foreign key, preventing an enrollment from ever producing more than one invoice.

Chapter 5

Data Requirements

5.1 Core Domain Entities

The following entities, drawn directly from the system's relational schema (`database/schema.sql`), define the minimum data SmartSchool must persist to satisfy the functional requirements in Chapter 3. Full column-level detail, the ER diagram, and the normalization rationale are presented in the companion SDD.

Entity	Required Attributes (minimum)
User	Id, name, email (unique), password hash, role (`admin`/`teacher`/`student`)
Student	Id, linked user, admission number, guardian contact, enrollment date
Course	Id, code, name, assigned teacher, credits, fee amount, term
Enrollment	Id, student, course, status, unique per (student, course)
Grade	Id, enrollment, assessment type, score (0–100), recorded by
Invoice	Id, student, enrollment (unique – one invoice per enrollment), amount (fee snapshot), method, status, issued/paid timestamps
Payroll Run	Id, period start/end, status, created by (planned feature)
Payslip	Id, payroll run, staff, base salary, deductions, net pay, status (planned feature)

5.2 Data Retention and Integrity

- An enrollment's status transitions (`active` → `completed`/`dropped`) shall be retained rather than deleted, preserving academic history.
- An invoice's `amount` shall remain fixed at the value snapshotted from the course fee at enrollment time, even if the course's fee later changes.
- Backups of the database shall be taken and verified by restore, per the Week 2 database-management build tasks.

Chapter 6
Traceability and Closing Notes

6.1 Traceability to the Software Design Document

Each functional requirement in Chapter 3 traces to a corresponding element of the SmartSchool SDD: authentication requirements map to the SDD's Auth Service design and sequence diagram (register/login); Academic and Finance requirements map to the microservice architecture diagram and the enrollment/invoice sequence diagram; Gateway requirements map to the Gateway component and routing table; non-functional requirements map to the SDD's Security, Performance, and Reliability sections.

6.2 Traceability to the Exam Mark Scheme

Exam Week	Marks	Requirements Covered
Week 1 – Environment & Auth	5	FR-AUTH-1 – FR-AUTH-3, NFR-SEC-2
Week 2 – Database	5	Chapter 5 (Data Requirements)
Week 3 – Microservices & Gateway	5	FR-GW-1 – FR-GW-4, FR-AUTH-6 – FR-AUTH-7
Week 4 – Messaging, Security & Performance	5	FR-ACAD-8, FR-FIN-1 – FR-FIN-2, NFR-SEC-1 – NFR-SEC-4, NFR-PERF-1 – NFR-PERF-2, NFR-REL-1 – NFR-REL-4
Week 5 – Module Feature, Testing & CI/CD	5	FR-HR-2 – FR-HR-4 or FR-FIN-5 (module feature), automated tests, CI pipeline (*planned*)
Week 6 – Integration & Documentation	5	Full `docker-compose` bring-up, this SRS/SDD pair, all UML diagrams (*planned*)

6.3 Open Items for the Group

- Decide which single feature the group will complete for Week 5's "one complete feature for your assigned module" task: payroll computation (HR), invoice payment/reconciliation (Finance), or a grade transcript (Academic).
- Set up the CI pipeline (GitHub Actions) and a first batch of automated tests for the chosen Week 5 feature.
- Decide, before Week 6, whether payment-gateway integration is added or explicitly documented as out of scope in the final submission.